P0593R3
Implicit creation of objects for low-level object manipulation

Published Proposal, 2019-01-18

This version:
http://wg21.link/p0593r3

Author:
Richard Smith (Google)

Former Author:
Ville Voutilainen

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes that objects of sufficiently trivial types be created on-demand as necessary within newly-allocated storage to give programs defined behavior.

Table of Contents

§ 1. Change history

Since [P0593R0]:

- Paper expanded from Ville's original call for solutions to a description of a proposed solution, based on SG12 discussion.

Since [P0593R1]:

Incorporated further SG12 feedback:

- An explicit syntactic marker is required to indicate that objects should be created. Existing obvious markers, such as the use of `malloc`, or simply performing member access on a union, suffice.
- Expand set of implicit lifetime types to require *either* a trivial default constructor *or* a trivial copy/move constructor, rather than requiring both.
  - Types with only a trivial default constructor may be suitable for member-by-member construction via class member access, even if the copy or move constructor is non-trivial.
  - Types with only a trivial copy/move constructor may be suitable for initialization by copying (for example) an on-disk representation into memory, even if the default constructor is non-trivial.
- Define the C standard library `memcpy` and `memmove` functions as triggering implicit object creation.
- Add description of suggested "typed" form of `std::bless`.

Since [P0593R2]:

- Removed union member access being sufficient to implicitly create objects based on objections from an implementer.

2. Motivating examples

2.1. Idiomatic C code as C++

Consider the following natural C program:

```
struct X { int a, b; };
X *make_x() {
  X *p = (X*)malloc(sizeof(struct X));
  p->a = 1;
  p->b = 2;
  return p;
}
```

When compiled with a C++ compiler, this code has undefined behavior, because `p->a` attempts to write to an `int` subobject of an `X` object, and this program never created either an `X` object nor an `int` subobject.

Per [intro.object]p1,

> An *object* is created by a definition, by a *new-expression*, when implicitly changing the active member of a union, or when a temporary object is created.

... and this program did none of these things.

2.2. Objects provided as byte representation

Suppose a C++ program is given a sequence of bytes (perhaps from disk or from a network), and it knows those bytes are a valid representation of type `T`. How can it efficiently obtain a `T *` that can be legitimately used to access the object?

Example: (many details omitted for brevity)

```
void process(Stream *stream) {
  unique_ptr<char[]> buffer = stream->read();
  if (buffer[0] == FOO)
    process_foo(reinterpret_cast<Foo*>(buffer.get())); // #1
  else
    process_bar(reinterpret_cast<Bar*>(buffer.get())); // #2
}
```

This code leads to undefined behavior today: within `Stream::read`, no `Foo` or `Bar` object is created, and so any attempt to access a `Foo` object through the `Foo*` produced by the cast at #1 would result in undefined behavior.

§ 2.3. Dynamic construction of arrays

Consider this program that attempts to implement a type like `std::vector` (with many details omitted for brevity):

```
template<typename T> struct Vec {
  char *buf = nullptr, *buf_end_size = nullptr, *buf_end_capacity = nullptr;
  void reserve(std::size_t n) {
    char *newbuf = (char*)::operator new(n * sizeof(T), std::align_val_t(alignof(T)));
    std::uninitialized_copy(begin(), end(), (T*)newbuf); // #a

    ::operator delete(buf, std::align_val_t(alignof(T)));
    buf_end_size = newbuf + sizeof(T) * size(); // #b
    buf_end_capacity = newbuf + sizeof(T) * n;  // #c
    buf = newbuf;
  }
  void push_back(T t) {
    if (buf_end_size == buf_end_capacity)
      reserve(std::max<std::size_t>(size() * 2, 1));
    new (buf_end_size) T(t);
    buf_end_size += sizeof(T); // #d
  }
  T *begin() { return (T*)buf; }
  T *end() { return (T*)buf_end_size; }
  std::size_t size() { return end() - begin(); } // #e
};
int main() {
  Vec<int> v;
  v.push_back(1);
  v.push_back(2);
  v.push_back(3);
  for (int n : v) { /*...*/ } // #f
}
```

In practice, this code works across a range of existing implementations, but according to the C++ object model, undefined behavior occurs at points #a, #b, #c, #d, and #e, because they attempt to perform pointer arithmetic on a region of allocated storage that does not contain an array object.

At locations #b, #c, and #d, the arithmetic is performed on a `char*`, and at locations #a, #e, and #f, the arithmetic is performed on a `T*`. Ideally, a solution to this problem would imbue both calculations with defined behavior.

§ 3. Approach

The above snippets have a common theme: they attempt to use objects that they never created. Indeed, there is a family of types for which programmers assume they do not need to explicitly create objects. We propose to identify these types, and carefully carve out rules that remove the need to explicitly create such objects, by instead creating them implicitly.

3.1. Affected types

If we are going to create objects automatically, we need a bare minimum of the following two properties for the type:

1) *Creating an instance of the type runs no code.* For class types, having a trivially default constructible type is often the right constraint. However, we should also consider cases where initially creating an object is non-trivial, but copying it (for instance, from an on-disk representation) is trivial.

2) *Destroying an instance of the type runs no code.* If the type maintains invariants, we should not be implicitly creating objects of that type.

Note that we're only interested in properties of the object itself here, not of its subobjects. In particular, the above two properties always hold for array types. While creating or destroying array elements might run code, creating the array object (without its elements) does not.

This suggests that the largest set of types we could apply this to is:

- Scalar types
- Array types (with any element type)
- Class types with a trivial destructor and a trivial constructor (of any kind)

(Put another way, we can apply this to all types other than function type, reference type, `void`, and class types where all constructors are non-trivial or where the destructor is non-trivial.)

We will call types that satisfy the above constraints *implicit lifetime types*.

3.2. When to create objects

In the above cases, it would be sufficient for `malloc` / `::operator new` to implicitly create sufficient objects to make the examples work. Imagine that `malloc` could "look into the future" and see how its storage would be used, and create the set of objects that the program would eventually need. If we somehow specified that `malloc` did this, the behavior of many C-style use cases would be defined.

On typical implementations, we can argue that this is not only natural, it is in some sense the status quo. Because the compiler typically does not make assumptions about what objects are created within the implementation of `malloc`, and because object creation itself typically has no effect on the physical machine, the compiler must generate code that would be correct if `malloc` did create that correct set of objects.

However, this is not always sufficient. An allocation from `malloc` may be sequentially used to store multiple different types, for instance by way of a memory pool that recycles the same allocation for multiple objects of the same size. It should be possible to grant such cases the same power to implicitly create objects as is de facto granted to `malloc`.

We could specify that implicit object creation happens automatically at any program point that relies on an object existing. This has a great deal of appeal: no explicit program action is ever required to create objects, and it directly describes a simple model where objects are not distinguished from the storage they occupy (this model gives the same results as C's "effective type" model in most cases). However, it also removes much of the power of scalar type-based alias analysis. The C committee has long been struggling with the conflict between their desire to support TBAA and their version of this rule, as exemplified by C's DR 236 ([C236]), which lists a "resolution" not reflected by the standard wording and that undesirably grants special powers to function call boundaries (this is one of at least four different and incompatible rules the C committee has at one point or another taken as the resolution to that defect). The lack of a reasonable resolution to these problems, despite them being known for nearly two decades, suggests that this is not a good path forward.

Therefore we propose the following rule:

> Some operations are described as implicitly creating objects within a specified region of storage. The abstract machine creates objects of implicit lifetime types within those regions of storage as needed to give the program defined behavior. For each operation that is specified as implicitly creating objects, that operation implicitly creates zero or more objects in its specified region of storage if doing so would give the program defined behavior. If no such sets of objects would give the program defined behavior, the behavior of the program is undefined.

The coherence of the above rule hinges on a key observation: changing the set of objects that are implicitly created can only change whether a particular program execution has defined behavior, not what the behavior is.

We propose that at minimum the following operations be specified as implicitly creating objects:

- Creation of an array of `char`, `unsigned char`, or `std::byte` implicitly creates objects within that array.
- A call to `malloc`, `calloc`, `realloc`, or any function named `operator new` or `operator new[]` implicitly creates objects in its returned storage.
- `std::allocator<T>::allocate` likewise implicitly creates objects in its returned storage; the allocator requirements should require other allocator implementations to do the same.
- A call to `memmove` behaves as if it

  1. copies the source storage to a temporary area
  2. implicitly creates objects in the destination storage, and then
  3. copies the temporary storage to the destination storage.

  This permits `memmove` to preserve the types of trivially-copyable objects, or to be used to reinterpret a byte representation of one object as that of another object.
- A call to `memcpy` behaves the same as a call to `memmove` except that it introduces an overlap restriction between the source and destination.
- A new barrier operation (distinct from `std::launder`, which does not create objects) should be introduced to the standard library, with semantics equivalent to a `memmove` with the same source and destination storage. As a strawman, we suggest:

```
// Requires: [start, (char*)start + length) denotes a region of allocated
// storage that is a subset of the region of storage reachable through start.
// Effects: implicitly creates objects within the denoted region.
void std::bless(void *start, size_t length);
```

In addition to the above, an implementation-defined set of non-standard memory allocation and mapping functions, such as `mmap` on POSIX systems and `VirtualAlloc` on Windows systems, should be specified as implicitly creating objects.

Note that a pointer `reinterpret_cast` is not considered sufficient to trigger implicit object creation.

§ 3.3. Type punning

We do not wish examples such as the following to become valid:

```
float do_bad_things(int n) {
  alignof(int) alignof(float)
    char buffer[max(sizeof(int), sizeof(float))];
  *(int*)buffer = n;      // #1
  std::bless(buffer, sizeof(buffer));
  return (*float*)buffer; // #2
}
```

```
float do_bad_things(int n) {
  union { int n; float f; } u;
  u.n = n;    // #1
  std::bless(&u, sizeof(u));
  return u.f; // #2
}
```

The proposed rule would permit an `int` object to spring into existence to make line #1 valid (in each case), and would permit a `float` object to likewise spring into existence to make line #2 valid.

However, these examples still do not have defined behavior under the proposed rule. The reason is a consequence of [basic.life]p4:

> The properties ascribed to objects and references throughout this document apply for a given object or reference only during its lifetime.

Specifically, the value held by an object is only stable throughout its lifetime. When the lifetime of the `int` object in line #1 ends (when its storage is reused by the `float` object in line #2), its value is gone. Symmetrically, when the `float` object is created, the object has an indeterminate value ([dcl.init]p12), and therefore any attempt to load its value results in undefined behavior.

Thus we retain the property (essential to modern scalar type-based alias analysis) that loads of some scalar type can be considered to not alias earlier stores of unrelated scalar types.

§ 3.4. Constant expressions

Constant expression evaluation is currently very conservative with regard to object creation. There is a tension here: on the one hand, constant expression evaluation gives us an opportunity to disallow runtime program semantics that we consider undesirable or problematic, and on the other hand, users strongly desire a full compile-time evaluation mechanism with the same semantics as the base language.

Following the existing conservatism in constant expression evaluation (eg, the disallowance of changing the active member of a union), we propose that the implicit creation of objects should *not* be performed during such evaluation.

§ 3.5. Pseudo-destructor calls

In the current C++ language rules, "pseudo-destructor" calls may be used in generic code to allow such code to be ambivalent as to whether an object is of class type:

```
template<typename T> void destroy(T *p) { p->~T(); }
```

When `T` is, say, `int`, the pseudo-destructor expression `p->~T()` is specified as having no effect. We believe this is an error: such an expression should have a lifetime effect, ending the lifetime of the `int` object. Likewise, calling a destructor of a class object should always end the lifetime of that object, regardless of whether the destructor is trivial.

This change improves the ability of static and dynamic analysis tools to reason about the lifetimes of C++ objects.

§ 3.6. Practical examples

```
std::vector<int> vi;
vi.reserve(4);
vi.push_back(1);
int *p = &vi.back();
vi.push_back(2);
vi.push_back(3);
int n = *p;
```

Within the implementation of `vector`, some storage is allocated to hold an array of up to 4 `int`s. Ignoring minor differences, there are two ways to create implicit objects to give the execution of this program defined behavior: within the allocated storage, either an `int[3]` object or an `int[4]` object is created. Both are correct interpretations of the program, and naturally both result in the same behavior. We can choose to view the program as being in the superposition of those two states. If we add a fourth `push_back` call to the program prior to the initialization of `n`, then only the `int[4]` interpretation remains valid.

```
unique_ptr<char[]> Stream::read() {
  // ... determine data size ...
  unique_ptr<char[]> buffer(new char[N]);
  // ... copy data into buffer ...
  return buffer;
}

void process(Stream *stream) {
  unique_ptr<char[]> buffer = stream->read();
  if (buffer[0] == FOO)
    process_foo(reinterpret_cast<Foo*>(buffer.get())); // #1
  else
    process_bar(reinterpret_cast<Bar*>(buffer.get())); // #2
}
```

Note the `new char[N]` implicitly creates objects within the allocated array. In this case, the program would have defined behavior if an object of type `Foo` or `Bar` (as appropriate for the content of the incoming data) were implicitly created *prior* to `Stream::read` populating its buffer. Therefore, regardless of which arm of the `if` is taken, there is a set of implicit objects sufficient to give the program defined behavior, and thus the behavior of the program is defined.

§ 4. Further work

§ 4.1. Direct object creation

In some cases it is desirable to change the dynamic type of existing storage while maintaining the object representation. If the destination type is an implicit lifetime type, this can be accomplished by usage of `std::bless` to change the type, followed by `std::launder` to acquire a pointer to the newly-created object. However, for expressivity and optimizability, a combined operation to create an object of implicit lifetime type in-place while preserving the object representation may be useful. Reviewers of a draft version of this paper have proposed:

```
// Effects: create an object of implicit lifetype type T in the storage
//          pointed to by T, while preserving the object representation.
template<typename T> T *bless(void *p);
```

Note that such an operation is not sufficient to implement *`node_handle`* ([P0083R3]) for map-like containers. *`node_handle`* requires the ability to take a `std::pair<const Key, Value>` and permit mutation of the `Key` portion (without destroying and recreating the `Key` object), even when `Key` is not an implicit lifetime type, so the above operation does not suffice. However, we could imagine extending its semantics to also permit conversions where each subobject of non-implicit-lifetime type in the destination corresponds to an object of the same type (ignoring cv-qualifications) in the source.

5. Acknowledgments

Thanks to Ville Voutilainen for raising this problem, and to the members of SG12 for discussing possible solutions.

References

Informative References

[C236]
Raymond Mak. C Defect Report #236: The interpretation of type based aliasing rule when applied to union objects or allocated objects.. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/dr_236.htm

[P0083R3]
Alan Talbot, Jonathan Wakely, Howard Hinnant, James Dennett. Splicing Maps and Sets (Revision 5). 24 June 2016. URL: https://wg21.link/p0083r3

[P0593R0]
Ville Voutilainen. What to do with buffers that are not arrays, and undefined behavior thereof?. 5 February 2017. URL: https://wg21.link/p0593r0

[P0593R1]
Richard Smith, Ville Voutilainen. Implicit creation of objects for low-level object manipulation. 16 October 2017. URL: https://wg21.link/p0593r1

[P0593R2]
Richard Smith. Implicit creation of objects for low-level object manipulation. 11 February 2018. URL: https://wg21.link/p0593r2